REPORT

REPORT ON THE SUBGROUP CHECK IMPLEMENTATION

Sebastian Ramacher
Omid Mir

05.10.2016

Review of the Tate Pairing-Based Subgroup Check Implementation

Omid Mir Sebastian Ramacher

October 20, 2026
AIT Austrian Institute of Technology

This report reviews the Tate-pairing subgroup check introduced in PR #7 of `https://github.com/TaceoLabs/babyjubjub-solidity`. The main goal was to verify the correctness of the implementation. During the review, additional performance opportunities were raised for consideration.

1 Correctness of the Subgroup Membership Test

Let Q denote the Baby Jubjub base-field modulus and R its prime subgroup order. The full group has order

$$\#E(\mathbb{F}_Q) = 8R.$$

Let G[1] be a generator of the full cyclic group. The desired subgroup is

$$\mathbb{G} = \langle 8 \cdot G \rangle,$$

which has prime order R. The following subgroup membership assumes that P is already a valid point on the Baby Jubjub curve, i.e., that $P \in E(\mathbb{F}_Q)$.

From [Kos22], it follows that for $P \in E(\mathbb{F}_Q)$ it holds that $P \in \mathbb{G}$ if and only if $e(T, P) = 1$ where $T = R \cdot G$ and e is the reduced Tate pairing since T has order 8 and generated the order-8 subgroup. After conversion to the Montgomery model, the point T is represented by

$u = 19799329160503878365519859265345525785148473002902384773314932802961476726446,$
$v = 17254532040196108728490891380973133526547459744854918265191993098218014369797.$

which are the values defined by `TATE_T_X` and `TATE_T_Y`.

[1] We use additive notation throughout this report.

2 Specialization of the Miller Loop

Because the first pairing argument T is fixed and has order 8, the Miller loop does not need to be implemented as a generic point-doubling loop. Only the tangent lines at T and $2 \cdot T$ are required. Specializing the generic Miller evaluation to these fixed points removes redundant operations.

Let the point at which the Miller function is evaluated be represented by projective Montgomery coordinates

$$u = \frac{U}{W}, v = \frac{V}{W}.$$

The tangent at T has affine equation $\ell_T(u, v) = v - \lambda_0 u + \lambda_0 T_x - T_y$. Using $\lambda_0 T_x - T_y = T_x$ and clearing the common denominator W, define $\ell_0 = V - \lambda_0 U + T_x W$. We have $2T = (1, y_{2\cdot T})$, where $y_{2\cdot T}$ corresponds to the constant defined `TATE_TWO_T_Y`.

The tangent at $2 \cdot T$ intersects the curve at $4 \cdot T = (0, 0)$. It therefore passes through the origin and can be written $v = y_{2\cdot T} u$. After clearing W, $\ell_1 = V - y_{2\cdot T} U$.

The specialized Miller value has the form

$$f = \frac{N}{D},$$

where $N = \ell_0^4 \ell_1^2$ and $D = WU(U - W)^4$.

Hence, the implementation represents an unrolled Miller loop under the constraint that T has order 8.

2.1 Final Exponentiation

Since final Tate exponent is $E = \frac{Q-1}{8}$. Hence, instead of exponentiating $\frac{N}{D}$, it can be rewritten as

$$f^E = (ND^7)^E.$$

Indeed,

$$\left(ND^7\right)^E = \left(\frac{N}{D}\right)^E D^{8E} = \left(\frac{N}{D}\right)^E D^{Q-1}.$$

For nonzero D, Fermat's theorem gives $D^{Q-1} = 1$. Therefore, the implementation presented in the PR uses these algebraic equivalences for improved efficiency.

Additionally, since $N = \ell_0^4 \ell_1^2 = (\ell_0^2 \ell_1)^2$. the exponentiations can be refactored into $ND^7 = ((D\ell_0)^2 \ell_1 D)^2 D$ which requires fewer multiplications than first computing N and D^7 independently. This change was recommended to be implemented to reduce GAS costs and the implementation was done as part of PR #10.

2.2 Final Exponentiation - Additional Considerations

In pairing implementations with non-trivial embedding degree, a common implementation technique is to factor the exponent into a ''hard'' and ''easy'' part [Sco+08]. The easy part captures powers of the field characteristic that can be handled by using the Frobenius endorphism instead of performing the actual exponentiation. Since the pairing in this case does not map into an extension field, the trick does not apply. The remaining factors of the exponent – the hard part – can be decomposed into vectorial addition chains [Oli81]. The exponent E can be written as $E = 2^{25} \cdot 12779 \cdot B$ where

$$B = 638078557170678430133064211230470412826523521781593725112608690 9.$$

For 12779 and B vector addition chains of length 4 and 38, respectively.

This change is only of benefit, if the GAS costs improve over the used of the precompiled `modExp` implementation. We have recommended to analyze the costs, but the cost analysis performed by Taceo came to the conclusion that it would not improve over the use of `modExp`.

3 Rewrite of the BabyJubJub Group Arithmetic

We were additionally tasked to review PR #13 and PR #14 which optimize the BabyJubJub arithmetic by reducing the cost of field inversion and scalar multiplication, respectively. PR #13 replaces Solidity-based exponentiation with the EVM modular exponentiation precompile and uses Montgomery's batch inversion technique in affine point addition. PR #14 replaces scalar multiplication with a Yul implementation of left-to-right binary double-and-add using extended twisted Edwards coordinates.

We establish the mathematical correctness of both optimizations and identify further opportunities to eliminate modular arithmetic instructions.

3.1 Correctness of PR #13

For the BabyJubJub base field $\mathbb{F}_q$ with prime q, we obtain

$$a^{-1} \equiv a^{q-2} \pmod{q}$$

for $a \neq 0$ using Fermat's little theorem. The original implementation evaluates this exponentiation using Solidity arithmetic. PR #13 delegates the same operation to the EVM modular exponentiation precompile. Hence, replacement is mathematically equivalent for nonzero field elements.

Affine twisted Edwards addition requires two inversions with denominators

$$u = 1 + dx_1x_2y_1y_2, \qquad v = 1 - dx_1x_2y_1y_2.$$

Instead of computing u^{-1} and v^{-1} separately, PR #13 computes

$$w = (uv)^{-1}$$

and obtains

$$u^{-1} = vw, \qquad v^{-1} = uw.$$

For valid BabyJubJub points, the completeness of the applicable twisted Edwards addition law guarantees nonzero denominators. Therefore, the transformation preserves the original group operation.

3.2 Correctness of PR #14

BabyJubJub is the twisted Edwards curve

$$ax^2 + y^2 = 1 + dx^2y^2$$

over $\mathbb{F}_q$, with $q < 2^{254}$. The assembly uses extended coordinates (X, Y, T, Z):

$$x = X/Z, \qquad y = Y/Z, \qquad XY = ZT \pmod{q}.$$

The initial accumulator $(x, y, xy, 1)$ satisfies this invariant.

For point doubling, the assembly implements

$$\begin{aligned} &A = X^2, && B = Y^2, && C = 2Z^2, \\ &D = aA, && E = (X+Y)^2 - A - B, \\ &G = D + B, && F = G - C, && H = D - B, \end{aligned}$$

followed by

$$(X', Y', T', Z') = (EF, GH, EH, FG).$$

These are the EFD `dbl-2008-hwcd`[2] [His+08] formulas.

For mixed point addition with a affine operand (x_2, y_2), the assembly precomputes

$$c_2 = dx_2y_2 \pmod{q}.$$

For an extended accumulator (X, Y, T, Z), it computes

$$\begin{aligned} &A = Xx_2, && B = Yy_2, && C = Tc_2, \\ &E = Xy_2 + Yx_2, \\ &F = Z - C, && G = Z + C, && H = B - aA, \end{aligned}$$

and returns

$$(X', Y', T', Z') = (EF, GH, EH, FG).$$

These are the EFD `madd-2008-hwcd` formulas.

Moving c_2 outside the scalar multiplication loop is correct because the affine operand is constant throughout the computation.

[2] `https://www.hyperelliptic.org/EFD/g1p/auto-twisted-extended-1.html`

3.3 Further Assembly Optimizations: Algebric Equivalences

The EVM computes

$$\texttt{mulmod}(u, v, q) = uv \bmod q$$

without requiring operands reduced mod$q$. Thus, modular reductions can be omitted when the resulting values are only consumed by `mulmod`. We only need to ensure that the operations do not overflow 256 bits before being consumed by `mulmod`. Hence, modular reductions can be delayed as long as $u \cdot v < 5q < 2^{256}$.[3] All inputs to `dbl` and `madd` are canonical representation in $[0, q)$.[4] Similarly, their outputs are also canonical because all output coordinates are computed using `mulmod`.

For the computation of the inputs for these calls to `mulmod`, we identify seven `addmod` instructions that can be replaced by `add` and `sub` and thereby saving up to 5 GAS per instruction.[5] Each replacement follows from one of the identities

$$u - v \equiv u + q - v \pmod{q},$$
$$u - v - w \equiv u + 2q - v - w \pmod{q}.$$

1. Doubling: computation of E. The original implementation computes

$$E = (X + Y)^2 - A - B \bmod q, \qquad A = X^2, \quad B = Y^2.$$

Original:

```
x3 := addmod(
    mulmod(x3, x3, Q),
    sub(Q, addmod(xx, yy, Q)),
    Q
)
```

Optimized:

```
x3 := sub(
    sub(
        add(
            mulmod(x3, x3, Q),
            Q2),
        xx),
    yy
)
```

[3] Note that $6q > 2^{256}$.

[4] With the only exception that before the first call to `dbl` this may need to be checked for potentially user-supplied points. In that case, the initial assignment of `x_res` and `y_res` needs to be reduced mod$q$.

[5] According to GAS costs from `https://ethereum.org/developers/docs/evm/opcodes/` as of October 16th, 2026.

where `Q2` represents a pre-computed $2 \cdot Q$.[6]

Since $0 \leq e, A, B < q$,

$$0 < e + 2q - A - B < 3q < 2^{256}.$$

2. Doubling: computation of F. The original expression is

$$F = G - C \bmod q, \qquad G = D + B, \quad C = 2Z^2 \bmod q.$$

Original:

```
z3 := addmod(
    y3,
    sub(Q, mulmod(mul(2,z1),z1,Q)),
    Q
)
```

Optimized:

```
z3 := sub(
    add(y3,Q),
    mulmod(mul(2,z1),z1,Q)
)
```

Here $0 \leq G < 2q$ and $0 \leq C < q$, hence

$$0 < G + q - C < 3q < 2^{256}.$$

3. Doubling: computation of H. The original expression is

$$H = D - B \bmod q.$$

Original:

```
xx := addmod(xx,sub(Q,yy),Q)
```

Optimized:

```
xx := sub(add(xx,Q),yy)
```

Since $0 \leq D, B < q$,

$$0 < D + q - B < 2q < 2^{256}.$$

[6] If $2 \cdot Q$ needs to be computed on the fly, that computation would add an additional call to `add` at the cost of 3 GAS saving 2 GAS in total according to `https://ethereum.org/developers/docs/evm/opcodes/`.

4. Mixed addition: computation of E. The original expression is

$$E = Xy_2 + Yx_2 \bmod q.$$

Original:

```
t3 := addmod(
   mulmod(x1,mload(0x20),Q),
   mulmod(y1,mload(0x00),Q),
   Q
)
```

Optimized:

```
t3 := add(
   mulmod(x1,mload(0x20),Q),
   mulmod(y1,mload(0x00),Q)
)
```

Since $0 \le a = Xy_2 \bmod q, b = Yx_2 \bmod q$

$$0 \le a + b < 2q < 2^{256}.$$

5. Mixed addition: computation of F. The original expression is

$$F = Z - C \bmod q, \qquad C = dTx_2y_2 \bmod q.$$

Original:

```
y3 := addmod(z1,sub(Q,x3),Q)
```

Optimized:

```
y3 := sub(add(z1,Q),x3)
```

Since $0 \le Z, C < q$,

$$0 < Z + q - C < 2q < 2^{256}.$$

6. Mixed addition: computation of H. The original expression is

$$H = B - aA \bmod q.$$

Original:

```
xx := addmod(
   yy,
   sub(Q,mulmod(A,xx,Q)),
   Q
)
```

Optimized:

```
xx := sub(
    add(yy,Q),
    mulmod(A,xx,Q)
)
```

Both B and $aA \bmod q$ are canonical residues. Consequently,

$$0 < B + q - (aA \bmod q) < 2q < 2^{256}.$$

3.4 Further Assembly Optimizations: CLZ

The loop

```
let i := 255
for {} iszero(and(shr(i, scalar), 1)) {} { i := sub(i, 1) }
```

computes the most significant bit (MSB) of `scalar` under the constraint that it is non-zero. Equivalently, the MSB of non-zero inputs can be computed by counting leading bits. EIP-7939 introduced the `clz` opcode and hence the MSB can be computed as

```
let i := sub(255, clz(scalar))
```

with constant GAS costs since Solidity version 0.8.31.

References

[His+08] Hüseyin Hisil, Kenneth Koon-Ho Wong, Gary Carter, and Ed Dawson. "Twisted Edwards Curves Revisited". In: *ASIACRYPT 2008*. Ed. by Josef Pieprzyk. Vol. 5350. LNCS. Springer, Berlin, Heidelberg, Dec. 2008, pp. 326–343. DOI: 10.1007/978-3-540-89255-7_20.

[Kos22] Dmitrii Koshelev. *Subgroup membership testing on elliptic curves via the Tate pairing*. Cryptology ePrint Archive, Report 2022/037. 2022. URL: https://eprint.iacr.org/2022/037.

[Oli81] Jorge Olivos. "On Vectorial Addition Chains". In: *J. Algorithms* 2.1 (1981), pp. 13–21. DOI: 10.1016/0196-6774(81)90003-1. URL: https://doi.org/10.1016/0196-6774(81)90003-1.

[Sco+08] Michael Scott, Naomi Benger, Manuel Charlemagne, Luis J. Dominguez Perez, and Ezekiel J. Kachisa. *On the final exponentiation for calculating pairings on ordinary elliptic curves*. Cryptology ePrint Archive, Report 2008/490. 2008. URL: https://eprint.iacr.org/2008/490.

AIT AUSTRIAN INSTITUTE OF TECHNOLOGY GMBH
Giefinggasse 4, 1210 Vienna, Austria

www.ait.ac.at

Sebastian Ramacher
Senior Scientist
Security & Communication Technologies
Center for Digital Safety & Security
sebastian.ramacher@ait.ac.at

Omid Mir
Scientist
Security & Communication Technologies
Center for Digital Safety & Security
omid.mir@ait.ac.at